

Command Injection, Path Traversal & File Upload

Command injection

An app that shells out (ping, convert, ffmpeg) and splices user input into the command:

```
# find the seam:
child_process.exec('ping -c 1 ' + host)

# inject:
host=127.0.0.1; whoami
127.0.0.1 & dir
$(cat /etc/passwd)
```

Detection automation

commix — the purpose-built scanner; understand its chunked-payload craft

Test every parameter that feeds a command-taking function

Defense

Never build commands from input; use parameterized exec (spawn/execFile with arg array)

Or strict allowlist input charset (hostnames only)

Egress-segment the shell host if unavoidable

Path traversal (LFI/TFI)

```
../../../../etc/passwd
..\..\..\Windows\win.ini
%2e%2e%2f%2e%2e%2f (encoded)
/FirstParam/../../../../etc/passwd (middle traversal)
```

Read app source, configs, or the OS – often escalates to RCE (LFI→phar/RFI→webshell).

Defense

Canonicalize (path.resolve) then ensure within base dir

Never join user input into paths raw; serve files by ID not path

Insecure file upload

Upload endpoint trusts filename+MIME → PHP/JSP/ASP uploaded and executed on server

```
# upload shell.php (mime-lied as image); access /uploads/shell.php
# -> RCE on the web user
```

Nastier variants

Double extensions (shell.php.jpg beat weak filters), case tricks (pHp), null bytes (exploit-era)

Content-type sniffing mismatch (image/x-... actually text)

Zip bomb / polyglots as DoS/unexpected-parse

Defense

Whitelist EXTENSION by signature (check Magic Bytes, not MIME only)

Store uploads OUTSIDE webroot; serve via controlled handler with Content-Disposition

AV-scan (ClamAV) + quarantined, random filenames; no user-controlled path

These three are the classic "I can get a shell" flaws. Their defense is structural: never splice data into files/commands; serve, don't execute.